Python Lists and Dictionaries

Store many things in one place, in order or by name.

Python | Beginner | Ages 9–16

InfiBee Coders

The Big Idea

- A list stores many items in order. A dictionary stores pairs: a key and its value.
- Use a list for things like scores or names. Use a dictionary when you look things up by name.
- Both can grow and shrink while your program runs.
- Together they hold almost all the data in real programs.

Lists

- Make one with square brackets: `fruits = ["apple", "banana"]`.
- Positions are called indexes and start at 0. `fruits[-1]` is the last item.
- `append(x)` adds to the end, `insert(i, x)` adds at a position.
- `remove(x)` deletes by value, `pop()` removes and returns the last item.
- `len()`, `sort()`, `reverse()`, `max()`, `min()`, `sum()` and `x in list` are very useful.
- Slicing: `scores[1:3]` gives items 1 and 2. The stop index is not included.

Dictionaries

- Make one with curly brackets: `pet = {"name": "Coco", "age": 3}`.
- Read a value by key: `pet["name"]`. Use `pet.get("colour", "unknown")` for a safe lookup.
- Add or change a value: `pet["age"] = 4`.
- `.keys()`, `.values()` and `.items()` give you the parts.
- Delete with `del pet["age"]` or `pet.pop("age")`.
- Keys must be unique, and are usually strings or numbers.

Choosing the Right One

- Order matters or you have many similar items: use a list.
- You want to look something up by a name: use a dictionary.
- A list of dictionaries is a great way to store many records, like students.
- A tuple like `(3, 4)` is a list that cannot be changed.

💻 Code Examples

List basics python

```python
fruits = ["apple", "banana", "mango"]
print(fruits[0])
print(fruits[-1])
fruits.append("kiwi")
print(len(fruits))
print(fruits)
```

▶ **Output:**

```
apple
mango
4
['apple', 'banana', 'mango', 'kiwi']
```

Index 0 is the first item, -1 is the last, and append adds kiwi to the end.

Sorting, maths and slicing python

```python
scores = [40, 95, 72, 88]
scores.sort()
print(scores)
print(max(scores), min(scores), sum(scores))
print(scores[1:3])
print(95 in scores)
```

▶ **Output:**

```
[40, 72, 88, 95]
95 40 295
[72, 88]
True
```

sort() puts numbers in order. in checks whether a value is in the list.

Add and remove python

```python
todo = ["homework", "play", "read"]
todo.insert(1, "lunch")
todo.remove("play")
last = todo.pop()
print(todo, last)
```

▶ **Output:**

```
['homework', 'lunch'] read
```

pop() removes the last item and gives it back.

Loop through a list python

```python
colours = ["red", "green", "blue"]
for c in colours:
    print(c.upper())
for i, c in enumerate(colours, start=1):
    print(i, c)
```

▶ **Output:**

```
RED
GREEN
BLUE
1 red
2 green
3 blue
```

enumerate gives you the number and the item together.

Dictionary basics python

```python
pet = {"name": "Coco", "age": 3}
print(pet["name"])
pet["age"] = 4
pet["colour"] = "brown"
print(pet)
print(pet.get("size", "unknown"))
```

▶ **Output:**

```
Coco
{'name': 'Coco', 'age': 4, 'colour': 'brown'}
unknown
```

Assigning to a new key adds it. get() avoids an error for a missing key.

Loop through a dictionary python

```python
marks = {"Aarav": 88, "Mia": 95, "Zoya": 72}
for name, score in marks.items():
    print(name, "scored", score)
print(list(marks.keys()))
print(sum(marks.values()) / len(marks))
```

▶ **Output:**

```
Aarav scored 88
Mia scored 95
Zoya scored 72
['Aarav', 'Mia', 'Zoya']
85.0
```

items() gives each key and value together.

A list of dictionaries python

```python
students = [
    {"name": "Aarav", "age": 12},
    {"name": "Mia", "age": 13},
]
for s in students:
    print(s["name"], "is", s["age"])
```

▶ **Output:**

```
Aarav is 12
Mia is 13
```

A common way to store records.

KeyError (missing key) python

```python
pet = {"name": "Coco"}
print(pet["age"])
```

▶ **Error message:**

```
KeyError: 'age'
```

Fix: check with `if "age" in pet:` or use pet.get("age").

IndexError (position does not exist) python

```python
colors = ["red", "blue"]
print(colors[2])
```

▶ **Error message:**

```
IndexError: list index out of range
```

A list with 2 items only has indexes 0 and 1.

📚 Key Terms

List
An ordered collection in square brackets.

Dictionary
A collection of key and value pairs.

Index
The position of an item. The first one is 0.

Key
The name used to look up a value.

Item
One value inside a list.

Value
The data stored under a key.

append()
Adds an item to the end of a list.

Tuple
A list that cannot be changed.

Slice
A part of a list, like items[1:3].

Iterate
Go through the items one by one.

🧮 Quick Reference

```
items = [1, 2, 3]
items[0]  items[-1]  items[1:3]
items.append(4)  items.remove(2)
len(items)  sum(items)  max(items)
```

```
d = {"a": 1, "b": 2}
d["a"]  d.get("z", 0)
d["c"] = 3
for k, v in d.items():
```

✏️ Worked Examples

Q: Make a list of 3 favourite foods and print the second one.
A: foods = ["pizza", "idli", "mango"]. Index 1 is the second item, so print(foods[1]) shows idli.

Q: Count how many times each letter appears in the word apple.
A: Make counts = {}. For each letter: counts[letter] = counts.get(letter, 0) + 1. Result: a 1, p 2, l 1, e 1.

Q: Store a person's name and age, then print a sentence.
A: person = {"name": "Mia", "age": 13}. print(f"{person['name']} is {person['age']}") shows Mia is 13.

🚫 Common Mistakes

- Thinking the first item is `[1]`. It is `[0]`.
- Using round brackets: `fruits(0)` instead of `fruits[0]`.
- Asking for a dictionary key that does not exist, which causes a KeyError.
- Changing a list while looping over it, which skips items.
- Forgetting that `sort()` changes the list and returns nothing.

👨‍👩‍👧 Parent Corner

- Use real lists: a shopping list, a playlist, a team. Ask for the first and last item.
- A dictionary is like a phone book: the name is the key and the number is the value.
- Ask your child to build a small contact book or a high score table.
- When they get a KeyError or IndexError, ask them to read the last line aloud.
- Praise tidy, descriptive names like `student_marks`.

🎯 Quick Check

1. What is the index of the first list item?

✅ 0.

2. How do you add an item to the end of a list?

✅ list.append(item).

3. How do you read a dictionary value?

✅ dict["key"] or dict.get("key").

4. What does len([4, 8, 15]) give?

✅ 3.

5. Which error means a dictionary key is missing?

✅ KeyError.